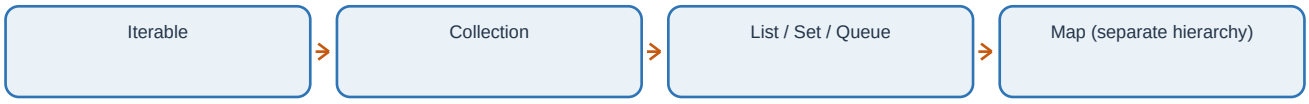
JAVA COLLECTIONS FRAMEWORK

Hierarchy • Lists • Sets • Queues • Maps • Sorting • Concurrency

Complete revision notebook
Definitions, comparison tables, hierarchy diagrams, Java examples, outputs and practical guidance.

COLLECTIONS FRAMEWORK AT A GLANCE

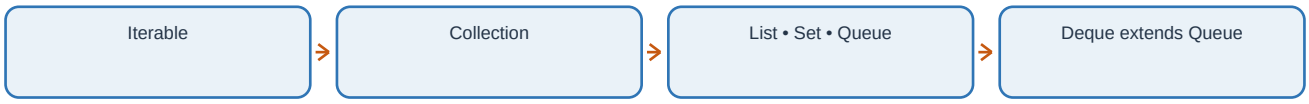


Version note: Examples use modern Java syntax and standard java.util / java.util.concurrent APIs. Some legacy classes are included for comparison and are generally not preferred in new code.

1. Collection Hierarchy, Iterable and Iterator

The Java Collections Framework is a set of interfaces, implementations and algorithms for storing and processing groups of objects. It reduces the need to build data structures from scratch.

SIMPLIFIED INTERFACE HIERARCHY



Important: Map is part of the Collections Framework but does not extend Collection. It stores key-value pairs.

Iterable and Iterator

• Iterable: provides an iterator and enables the enhanced for loop.

• Iterator: moves through elements using hasNext() and next(); remove() is optional and removes the last returned element when supported.

```
List<String> names = new ArrayList<>(List.of("Asha", "Ravi"));

Iterator<String> it = names.iterator();
while (it.hasNext()) {
    String name = it.next();
    System.out.println(name);
}
```

Output

```
Asha\nRavi
```

• Do not call next() without checking hasNext(); it may throw NoSuchElementException.

• Use Iterator.remove() for safe removal during iterator traversal when supported.

• An enhanced for loop is convenient for reading, but it does not expose the iterator's remove method.

2. Collection vs Collections

`Collection` is the root interface for groups of elements (except Map). `Collections` is a utility class with static methods that operate on collections.

Collection	Collections
Interface	Utility class
Implemented by List, Set, Queue, etc.	Provides sort, reverse, shuffle, min, max, frequency, unmodifiable wrappers, synchronized wrappers, etc.
Represents a group of elements	Provides reusable algorithms and wrappers

```
List<Integer> values = new ArrayList<>(List.of(4, 1, 3));
Collections.sort(values);
System.out.println(values);
Collections.reverse(values);
System.out.println(values);
```

Output

```
[1, 3, 4]\n[4, 3, 1]
```

Note: List.of(...) creates an unmodifiable list. The example wraps it in ArrayList so that sorting and reversing are allowed.

3. ArrayList, LinkedList, Vector and Stack

These are List implementations with different internal structures and performance trade-offs.

Class	Structure / behavior	Typical use
ArrayList	Resizable array; fast indexed access; middle insert/remove shifts elements	Default general-purpose list
LinkedList	Doubly linked nodes; indexed access requires traversal; efficient operations at ends	Queue/deque operations; rarely best for indexed lists
Vector	Legacy growable array; individual methods synchronized	Legacy APIs; usually prefer ArrayList or concurrent design
Stack	Legacy Vector subclass; LIFO methods	Prefer Deque / ArrayDeque

```
List<String> list = new ArrayList<>();
list.add("Java");
list.add("SQL");
System.out.println(list.get(0)); // Java

Deque<String> stack = new ArrayDeque<>();
stack.push("first");
stack.push("second");
System.out.println(stack.pop()); // second
```

Complexity intuition

• ArrayList: get/set by index are O(1) on average; appending is amortized O(1); insertion/removal near the front or middle is O(n).

• LinkedList: adding/removing at either end is O(1); locating an indexed position is O(n).

• Vector's synchronization does not make a multi-step sequence automatically atomic.

Recommendation: choose ArrayList for most list needs. Use ArrayDeque for stack/queue behavior. Choose LinkedList only when its node-based operations fit the access pattern.

4. HashSet, LinkedHashSet and TreeSet

A Set stores unique elements. Adding an element already present does not increase the set size.

Set	Ordering	Typical performance
HashSet	No guaranteed iteration order	Expected O(1) add, remove, contains
LinkedHashSet	Insertion order	Expected O(1) basic operations with extra link overhead
TreeSet	Sorted order (natural or comparator)	O(log n) basic operations

```
Set<String> a = new HashSet<>();
a.add("red"); a.add("blue"); a.add("red");
System.out.println(a.size()); // 2

Set<Integer> sorted = new TreeSet<>(List.of(5, 2, 4));
System.out.println(sorted); // [2, 4, 5]
```

Rules and cautions

• Hash-based sets depend on correct equals() and hashCode() implementations.

• TreeSet uses comparison (Comparable or Comparator) to determine ordering and uniqueness; compare(a,b) == 0 treats values as the same set element.

• Do not mutate fields used in hashing or ordering while an element is stored in a set.

Use cases: HashSet for membership checks, LinkedHashSet for de-duplication while preserving encounter order, TreeSet for sorted unique values and range queries.

5. Queue, PriorityQueue, Deque and ArrayDeque

Queue represents elements waiting to be processed. It commonly follows FIFO (first in, first out), although PriorityQueue orders by priority rather than insertion time.

Queue method pairs

Operation	Throws on failure	Special result on failure
Insert	add(e)	offer(e) returns false
Remove head	remove()	poll() returns null
Examine head	element()	peek() returns null

• PriorityQueue: head is the least element by natural ordering or supplied comparator by default. Iterating it does not promise sorted traversal; repeatedly poll() to retrieve priority order.

• Deque: double-ended queue; supports insertion/removal at both ends. It can act as FIFO queue or LIFO stack.

• ArrayDeque: resizable-array Deque; generally preferred over Stack and often over LinkedList for queue/stack operations. It does not permit null elements.

```
Queue<Integer> q = new ArrayDeque<>();
q.offer(10); q.offer(20);
System.out.println(q.poll()); // 10

PriorityQueue<Integer> pq = new PriorityQueue<>();
pq.offer(30); pq.offer(10); pq.offer(20);
System.out.println(pq.poll()); // 10

Deque<String> d = new ArrayDeque<>();
d.push("A"); d.push("B");
System.out.println(d.pop()); // B
```

Practical use: task scheduling, breadth-first search, undo stacks, parsers and work queues.

6. Map, HashMap, LinkedHashMap and Hashtable

Map stores key-value pairs. Keys are unique; a key maps to at most one value. Values may repeat. Map is not a subtype of Collection.

Map	Ordering / behavior	Notes
HashMap	No guaranteed order	Expected O(1) get/put; allows one null key and null values; not thread-safe
LinkedHashMap	Insertion order by default; can be configured for access order	Useful for predictable iteration and LRU-style caches
Hashtable	No guaranteed order	Legacy synchronized map; no null keys or values

```
Map<String, Integer> marks = new HashMap<>();
marks.put("Asha", 90);
marks.put("Ravi", 85);
System.out.println(marks.get("Asha")); // 90
System.out.println(marks.containsKey("Ravi")); // true

for (Map.Entry<String, Integer> e : marks.entrySet()) {
    System.out.println(e.getKey() + ": " + e.getValue());
}
```

Useful Map methods

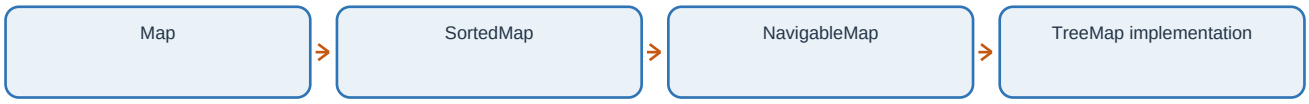
- put, get, getOrDefault, containsKey, remove, putIfAbsent, replace, keySet, values, entrySet.
- computeIfAbsent is useful for creating a value only when a key is missing.
- Do not assume HashMap iteration order. Use LinkedHashMap if order matters.

Hashtable vs HashMap: Hashtable is legacy synchronized and disallows null; HashMap is generally preferred for single-threaded use. For concurrent access, consider ConcurrentHashMap.

7. TreeMap, SortedMap and NavigableMap

TreeMap stores keys in sorted order using natural ordering or a Comparator. It implements NavigableMap, which extends SortedMap, which extends Map.

MAP INTERFACE RELATIONSHIP



• SortedMap: exposes firstKey(), lastKey(), comparator(), headMap(), tailMap() and subMap().

• NavigableMap: adds lower/floor/ceiling/higher key searches, descendingMap(), pollFirstEntry() and pollLastEntry().

• TreeMap basic get/put/remove operations are O(log n). Null keys are generally not allowed with natural ordering; comparator behavior determines valid keys.

```
NavigableMap<Integer, String> map = new TreeMap<>();
map.put(20, "B"); map.put(10, "A"); map.put(30, "C");

System.out.println(map.firstKey());       // 10
System.out.println(map.ceilingKey(21));   // 30
System.out.println(map.floorKey(21));     // 20
System.out.println(map.subMap(10, true, 30, false));
```

Output

```
10\n30\n20\n{10=A, 20=B}
```

Range view caution: headMap, tailMap and subMap return backed views. Changes through a view can affect the original map.

8. ConcurrentHashMap, BlockingQueue and Copy-on-Write

The java.util.concurrent package provides collections designed for concurrent programs, where multiple threads access shared data.

• ConcurrentHashMap: thread-safe hash map supporting concurrent retrievals and updates. It does not allow null keys or values. Use atomic methods such as putIfAbsent, compute and merge for compound updates.

• BlockingQueue: queue operations can wait for space or elements. Common implementations include ArrayBlockingQueue and LinkedBlockingQueue.

• CopyOnWriteArrayList / CopyOnWriteArraySet: each write creates a fresh backing array. Iterators traverse a snapshot and do not reflect later modifications.

```
BlockingQueue<String> jobs = new ArrayBlockingQueue<>(10);

// Producer thread:
jobs.put("email");

// Consumer thread:
String job = jobs.take();
System.out.println(job); // email

ConcurrentMap<String, Integer> counts = new ConcurrentHashMap<>();
counts.merge("java", 1, Integer::sum);
```

When to choose

• ConcurrentHashMap: shared key-value state with frequent concurrent reads and updates.

• BlockingQueue: producer-consumer pipelines and bounded work buffers.

• Copy-on-write: many reads/iterations and very few writes, such as listener registries.

Important: thread-safe individual operations do not automatically make a multi-operation business transaction atomic. Use the relevant atomic methods or explicit coordination.

9. Comparable vs Comparator

Both define ordering, but they place the ordering rule in different locations.

Comparable	Comparator
Implemented by the class being compared	Separate object or lambda defines comparison
compareTo(other)	compare(a, b)
Usually natural/default ordering	Alternative or custom orderings

```
class Student implements Comparable<Student> {
    String name;
    int marks;
    Student(String n, int m) { name = n; marks = m; }

    public int compareTo(Student other) {
        return Integer.compare(this.marks, other.marks);
    }
}
```

```
List<Student> students = new ArrayList<>();
students.add(new Student("Ravi", 80));
students.add(new Student("Asha", 95));

students.sort(Comparator.comparing(s -> s.name));
students.sort(Comparator.comparingInt(s -> s.marks).reversed());
```

Comparator helpers

• Comparator.comparing(...), comparingInt(...), thenComparing(...), reversed(), nullsFirst(...) and nullsLast(...).

• Prefer Integer.compare(a, b) over a - b to avoid integer overflow.

• For TreeSet/TreeMap, comparison result zero determines key/element equivalence, so ordering should be consistent with equals when possible.

Use case: Comparable for a natural order such as product ID; Comparator for sorting the same products by price, name or rating.

10. equals/hashCode Contract and Hash Collisions

Hash-based collections use hashCode() to find a bucket and equals() to identify a matching key or element within that bucket.

Contract essentials

• If a.equals(b) is true, a.hashCode() must equal b.hashCode().

• If two objects have the same hash code, they do not have to be equal. Different objects can collide.

• equals() should be reflexive, symmetric, transitive and consistent; comparing to null returns false.

• If fields used by equals/hashCode change while an object is a HashMap key or HashSet element, lookup/removal may fail.

```
class Employee {
    private final int id;
    Employee(int id) { this.id = id; }

    @Override
    public boolean equals(Object obj) {
        if (this == obj) return true;
        if (!(obj instanceof Employee other)) return false;
        return id == other.id;
    }

    @Override
    public int hashCode() {
        return Integer.hashCode(id);
    }
}
```

Hash collision

A collision occurs when different keys produce the same hash value or map to the same bucket. A correct collection still checks equality, so collisions do not mean keys are treated as equal. HashMap handles collisions internally; performance can degrade when many keys collide, though modern implementations can treeify heavily populated bins under qualifying conditions.

Best practice: use immutable key fields, implement equals and hashCode together, and use Objects.equals / Objects.hash where appropriate. Records automatically provide value-based equals and hashCode for their components.

11. Fail-Fast, Immutable and Unmodifiable Collections

Fail-fast iterator: many standard collection iterators may throw ConcurrentModificationException if the collection is structurally modified outside the iterator during iteration. This is best-effort bug detection, not a thread-safety guarantee.

```
List<String> names = new ArrayList<>(List.of("A", "B", "C"));

for (String name : names) {
    if (name.equals("B")) {
        // names.remove(name); // may throw ConcurrentModificationException
    }
}

// Safe iterator-based removal:
Iterator<String> it = names.iterator();
while (it.hasNext()) {
    if (it.next().equals("B")) it.remove();
}
```

Immutable vs unmodifiable

• Unmodifiable view: blocks modification through that reference, but changes to the backing collection may still appear in the view.

• Immutable collection: its contents cannot be changed after creation; a true immutable object should not expose mutable internal state.

• Collections.unmodifiableList(list) returns an unmodifiable view, not a defensive immutable copy.

• List.of(...) and Set.of(...) create unmodifiable collections; they reject nulls. Map.of(...) creates an unmodifiable map and rejects null keys/values.

```
List<String> source = new ArrayList<>(List.of("A"));
List<String> view = Collections.unmodifiableList(source);
source.add("B");
System.out.println(view); // [A, B] — view reflects backing change

List<String> fixed = List.of("X", "Y");
// fixed.add("Z"); // UnsupportedOperationException
```

Safe copy: List.copyOf(source) creates an unmodifiable snapshot of the source's elements (and rejects null elements). It is shallow: mutable elements themselves are not made immutable.

12. Final Revision Sheet & Practice

Quick memory points

• Iterable provides Iterator; Iterator traverses using hasNext() and next().

• Collection is an interface; Collections is a utility class.

• ArrayList is the usual list default; ArrayDeque is preferred for stack/queue use.

• HashSet is unordered; LinkedHashSet preserves insertion order; TreeSet sorts.

• PriorityQueue retrieves by priority; Deque works at both ends.

• HashMap has no order guarantee; LinkedHashMap preserves order; TreeMap sorts keys.

• ConcurrentHashMap supports concurrent map operations; BlockingQueue coordinates producer-consumer threads.

• Comparable defines natural order; Comparator defines external/custom order.

• Equal objects must have equal hash codes; collisions are allowed and resolved with equals().

• Fail-fast is best-effort; unmodifiable view is not necessarily immutable.

Practice questions

1. Does Map extend Collection?
2. What does Iterable provide?
3. Which list is usually best for indexed access?
4. Which Set preserves insertion order?
5. Does PriorityQueue iteration return sorted order?
6. Which Map sorts keys?
7. What is the difference between ConcurrentHashMap and Hashtable?
8. When should BlockingQueue be used?
9. What is the equals/hashCode contract?
10. Does Collections.unmodifiableList make a defensive copy?
11. What is a fail-fast iterator?

Answer key

1. No. 2. Iterator and enhanced-for support. 3. ArrayList. 4. LinkedHashSet. 5. No; repeatedly poll(). 6. TreeMap. 7. ConcurrentHashMap is designed for modern concurrent access and rejects null; Hashtable is legacy synchronized. 8. Producer-consumer coordination. 9. Equal objects must share a hash code. 10. No, it is a view. 11. An iterator that may detect structural modification and throw ConcurrentModificationException.

Study tip: practise choosing a collection from the operation you need—index access, uniqueness, sorted order, FIFO, LIFO, key lookup or concurrent coordination—rather than memorizing class names alone.